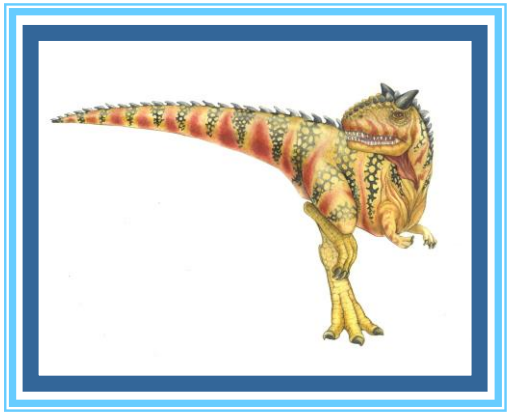
Chapter 6: Deadlocks

فصل ششم: بن بست



Silberschatz, Galvin and Gagne ©2013

Outline

- مدل سیستم (System Model)
- انواع منابع
- تعریف و ویژگی‌های Deadlock
- روش‌های برخورد با Deadlock
- پیشگیری (Prevention)
- اجتناب (Avoidance)
- تشخیص (Detection)
- بازیابی (Recovery)



Chapter Objectives اهداف فصل

- در پایان این فصل دانشجو می‌تواند:
- توضیح دهد deadlock چگونه رخ می‌دهد
- چهار شرط لازم deadlock را نام ببرد
- deadlock را در گراف تخصیص منابع تشخیص دهد
- روش‌های جلوگیری و اجتناب را مقایسه کند
- الگوریتم بانکدار (Banker’s Algorithm) را اجرا کند
- روش‌های تشخیص و بازیابی را تحلیل کند



System Model

- سیستم شامل منابع (Resources) است.
- انواع منابع:
  - CPU
  - حافظه
  - دستگاه‌های I/O
- هر نوع منبع Riدارای تعدادی Wi نمونه است.
- مراحل استفاده از منابع توسط فرآیندها عبارت است از:

1) request(درخواست): تا زمانی‌که درخواست قبول شود و منبع را دریافت کند، فرآیند منتظر خواهد ماند.
2) use(استفاده)
3) release(آزادسازی)

- deadlockمعمولاً زمانی رخ می‌دهد که مرحله release انجام **نشود.**



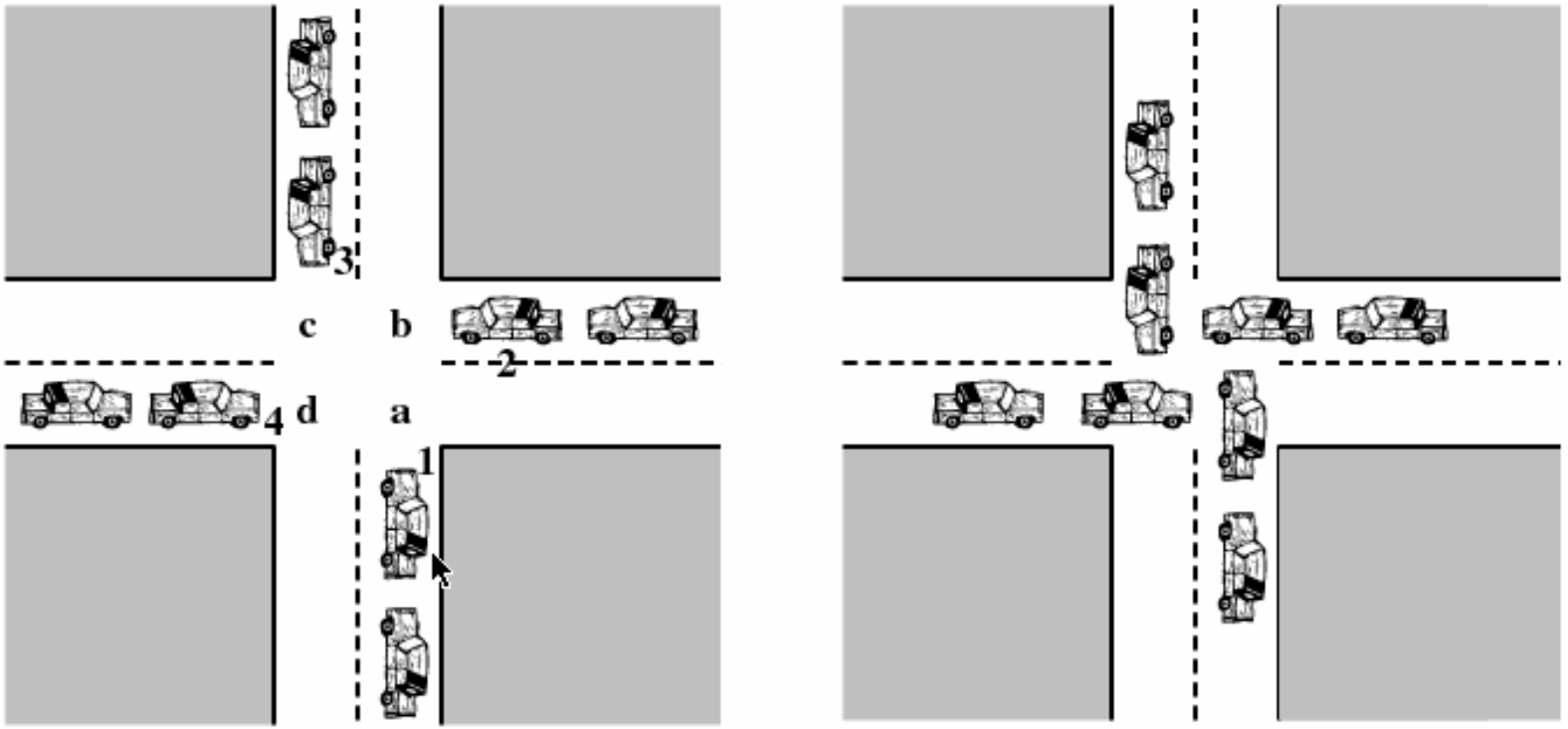
بن بست

- Deadlock وضعیتی است که در آن چند فرآیند به‌صورت دائمی منتظر یکدیگر می‌مانند و هیچ‌کدام قادر به ادامه اجرا نیستند.
- در بن‌بست، هر فرآیند منبعی را در اختیار دارد و منتظر منبعی است که توسط فرآیند دیگر نگه داشته شده است.
- deadlock یک مفهوم مدیریت منابع است، نه صرفاً «منتظر ماندن». کلید فهم بن بست منبع(resource) است.
- برای بن بست راه حل کارآمدی وجود ندارد.
- تمام بن‌بست‌ها با نیازهای متضاد دو فرایند یا بیشتر، برای منابع همراه هستند.



مثال بن بست

- یک مثال بن بست، ترافیک است.در رانندگی قانون این است که هر خودرو باید تسلیم خودروی سمت راست باشد.در این صورت در حالت زیر چه رخ میدهد؟



الف)امکان بن بست

ب) بن بست

مثال بن بست در فرایندها

- ساده ترین نوع بن بست زمانی اتفاق می افتد که دو فرایند مجزا و همزمان منتظر بدست آوردن منبعی هستند که در اختیار دیگریست و خود نیز منابعشان را تا اتمام فرایند آزاد نمیکنند.

Process P1

```
Get A
Get B
Release A
Release B
.
.
```

Process P2

```
Get B
Get A
Release B
Release A
.
.
```



Types of Resources in Operating Systems

- منابع نقش تعیین کننده ای در بن بست دارند.
- منابع به دو دسته تقسیم میشوند:
  - منابع قابل استفاده مجدد (Reusable Resources)
  - منابع مصرف شدنی یا غیر قابل استفاده مجدد



منابع قابل استفاده مجدد Reusable Resources

- منابعی هستند که در هر لحظه از زمان تنها توسط یک فرایند قابل استفاده اند و استفاده از آنها موجب به پایان رسیدن آنها نمیشود(بدون آسیب دیدن آزاد میشوند)
- این منابع پس از استفاده، قابل آزادسازی و استفاده‌ی دوباره توسط فرآیندهای دیگر هستند.
- ویژگی‌ها:
  - منبع پس از استفاده از بین نمی‌رود
  - فقط برای مدت محدودی در اختیار یک فرآیند است
  - پس از release دوباره به سیستم بازمی‌گردد
- مثال‌ها:
  - CPU
  - Main Memory (RAM)
  - Files
  - I/O Devices مثل پرینتر یا دیسک
  - Semaphores, Mutexes
- بیشتر موارد Deadlock کلاسیک مربوط به منابع قابل استفاده‌ی مجدد است.
- بن بست زمانی رخ میدهد که یک فرایند منابع را نگه دارد و منبع دیگری درخواست کند.



مثالی از بن بست منابع قابل استفاده مجدد

- فرض کنید فضای حافظه که میتواند به فرایند ها تخصیص یابد 200 KB باشد.در این صورت ترتیب زیر موجب بروز بن بست میشود.

P1	P2
. . .	. . .
Request 80 Kbytes;	Request 70 Kbytes;
. . .	. . .
Request 60 Kbytes;	Request 80 Kbytes;



منابع غیرقابل استفاده‌ی مجدد یا مصرف شدنی
Consumable Resources یا Non-reusable Resources

- این منابع پس از مصرف، از بین می‌روند و دیگر قابل بازگشت نیستند.
- **ویژگی‌ها:**
  - پس از استفاده نابود می‌شوند
  - قابل آزادسازی به سیستم نیستند
  - معمولاً به‌صورت داده یا سیگنال هستند
- **مثال‌ها:**
  - Messages در سیستم‌های Message Passing
  - Signals
  - Interrupts
  - Packets در شبکه
- این نوع منابع معمولاً باعث Starvation می‌شوند، نه Deadlock کلاسیک.(غیرمستقیم)



مثالی از بن بست منابع مصرف شدنی:

- در صورتیکه عمل Receive مسدود کننده باشد بن بست منطقی اتفاق می افتد.
- این وضعیت بن‌بست منطقی در سطح برنامه است،نه بن‌بست منابع در سطح سیستم‌عامل.

```
P1
...
Receive(P2);
...
Send(P2, M1);
```

```
P2
...
Receive(P1);
...
Send(P1, M2);
```



بن‌بست سیستم‌عاملی و بن‌بست منطقی

- بن‌بست سیستم‌عاملی (OS Deadlock):
  - مربوط به منابع قابل استفاده مجدد مانند CPU، حافظه و پرینتر
  - منابع توسط فرآیندها نگه داشته می‌شوند
  - سیستم‌عامل می‌تواند آن را تشخیص دهد و مدیریت کند
  - مثال: هر فرآیند منبعی را گرفته و منتظر منبع فرآیند دیگر است
- بن‌بست منطقی (Logical Deadlock):
  - مربوط به منطق نادرست برنامه یا پروتکل ارتباطی
  - منبع واقعی توسط سیستم‌عامل نگه داشته نمی‌شود
  - سیستم‌عامل قادر به تشخیص یا رفع آن نیست وفقط با اصلاح برنامه قابل حل است
  - مثال: دو فرآیند که هر دو منتظر دریافت پیام از یکدیگر هستند
- Deadlock-like Situation:
  - رفتاری شبیه بن‌بست دارد
  - برنامه متوقف می‌شود اما شرایط رسمی Deadlock برقرار نیست
  - اغلب برای توصیف بن‌بست‌های منطقی استفاده می‌شود



چهار شرط لازم برای Deadlock

- بن‌بست فقط زمانی رخ می‌دهد که هر چهار شرط همزمان برقرار باشند:

1) انحصارمتقابل **Mutual Exclusion**:فقط یک نخ می‌تواند از یک منبع استفاده کند.

2) نگهداری و انتظار **Hold and Wait**: نخ حداقل یک منبع دارد ومنتظر منابع دیگر است.

3) انحصاری بودن **No Preemption**: منبع را نمی‌توان به زور از نخ گرفت.

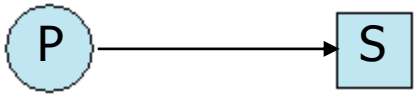
4) انتظار چرخشی**Circular Wait**: زنجیره‌ای دایره‌ای از انتظار وجود دارد.

- اگر حتی یکی از این شرایط حذف شود : بن‌بست رخ نمی‌دهد.



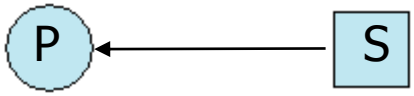
Resource-Allocation Graph گراف تخصیص منابع

- این گراف ابزار اصلی تحلیل deadlock است.
- گراف تخصیص منابع یک گراف جهت دار است که نحوه تخصیص منابع به فرایند ها را در هر لحظه از زمان نشان میدهد.
- برای تشخیص بن بست باید گراف تخصیص منابع را بعد از هر درخواست، هر تخصیص، یا هر ترخیص به روز کرد.
- در گراف ، منابع را با مربع (تعداد در آن مشخص)و فرایند را با دایره نشان میدهیم.
- وجود دور یا چرخه در گراف بیانگر بروز بن بست است. و اگر گراف تخصیص منابع فاقد چرخه باشد، حالت بن بست وجود ندارد.
- درخواست: Pi → Rj



فرایند P در انتظار منبع S است

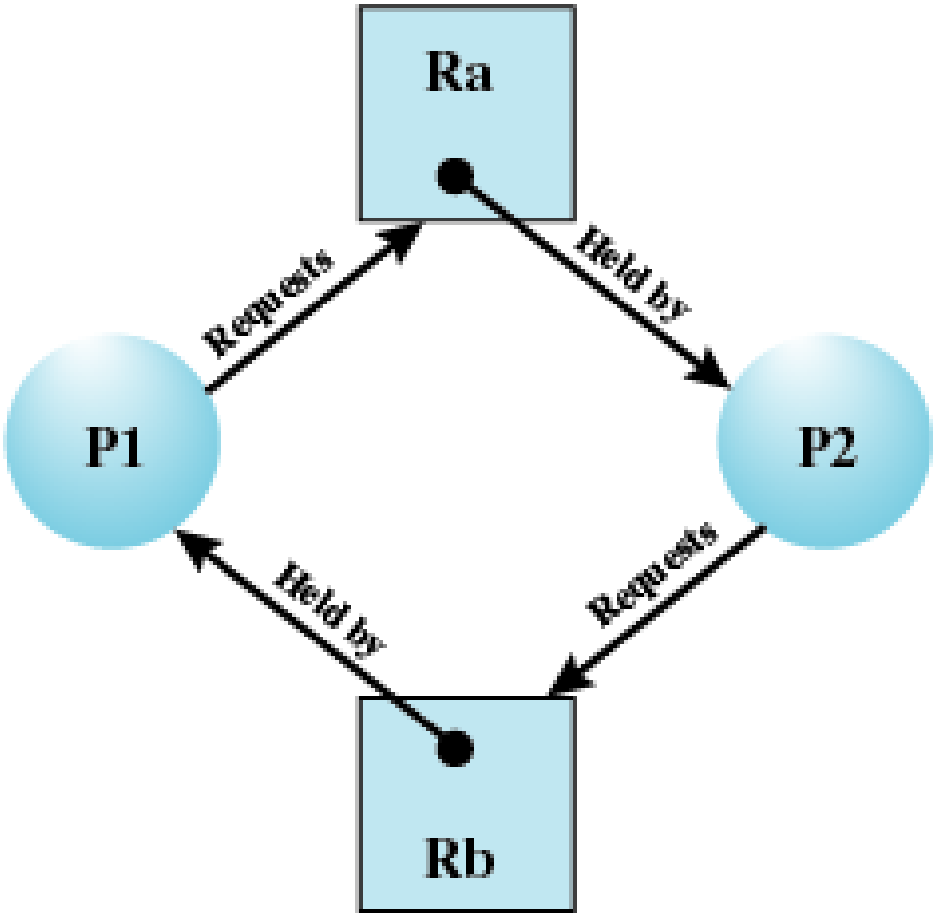
- تخصیص: Rj → Pi



فرایند P منبع S را در اختیار دارد

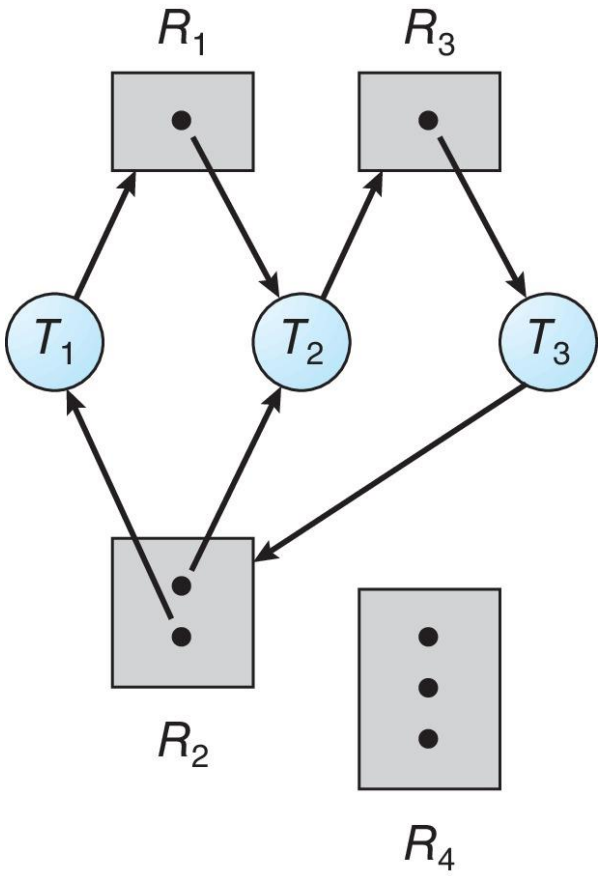


انتظار چرخشی



Graph with a Deadlock

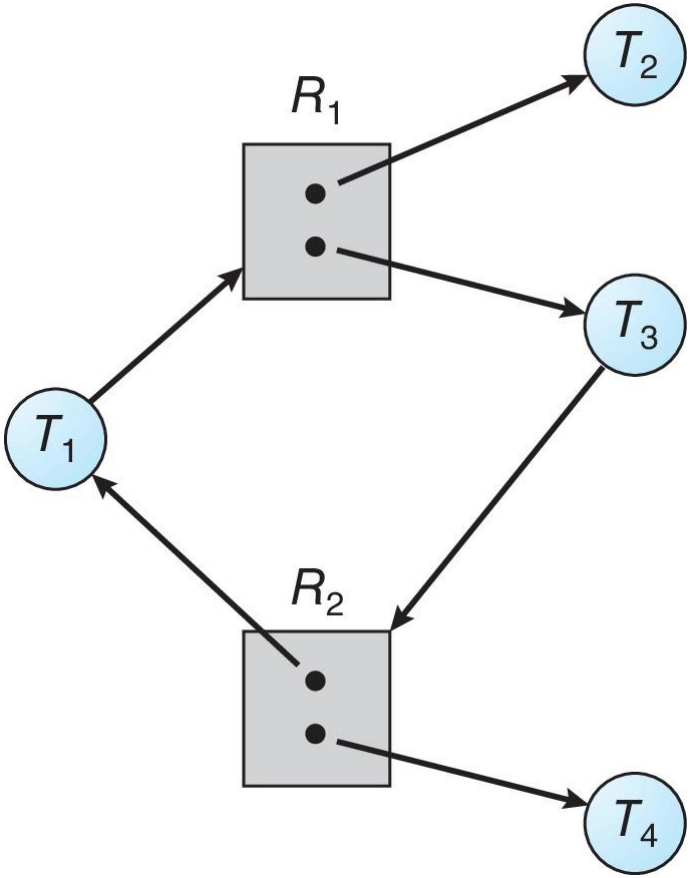
- وجود چرخه در گراف (وقتی هر منبع یک نمونه دارد) → بن‌بست قطعی است.





Graph with a Cycle But no Deadlock

- اگر منابع چند نمونه‌ای باشند، وجود چرخه الزاماً به معنی deadlock نیست.





معیار تشخیص اولیه در گراف تخصیص منابع

- اگر گراف تخصیص منابع چرخه یا حلقه ندارد.
  - سیستم بن بست ندارد.

- اگر گراف تخصیص منابع حلقه دارد.
  - اگر از هر نوع منبع درگیر در حلقه فقط یک نمونه داریم، بن بست رخ داده است.
  - اگر از هر نوع منبع درگیر در حلقه چندین نمونه داریم، احتمال وقوع بن بست وجود دارد.



روشهای برخورد با بن‌بست

1) نادیده گرفتن بن‌بست(شترمرغ)

- فرض میشود هیچگاه بن‌بست رخ نمی‌دهد، هیچ عملی در مقابل بن‌بست انجام نمی‌شود، پس از وقوع بن‌بست ری‌استارت می‌شود.

2) پیشگیری یا جلوگیری از بن بست(Prevention)

- سیستم طوری طراحی شود که از قبل امکان بن‌بست از بین برود، یعنی تضمین کنیم که حداقل یکی از چهار شرط بن‌بست رخ ندهد.

3) اجتناب از بن بست (Avoidance)

- در مورد درخواستهای منبع طوری رفتار شود که حداقل یکی از چهار شرط بن بست رخ ندهد.

4) کشف بن بست (Detection)

- هرجا که ممکن باشد، منابع درخواستی به فرایندها داده میشود. سیستم عامل به طور متناوب الگوریتمی را دنبال میکندتا وجود شرط انتظار چرخشی را کشف کند.



Deadlock Prevention پیشگیری از بن‌بست

هدف: **نقض حداقل یکی از چهار شرط**

1) Mutual Exclusion که قابل حذف نیست بعضی از منابع ذاتاً انحصاری هستند.

2) Hold and Wait :جلوگیری از این که فرآیند هم منابعی را نگه دارد و هم منتظر منابع دیگر باشد.

- روش‌ها:
  - درخواست همه منابع یکجا: فرآیند فقط وقتی شروع به اجرا کند که بتواند تمام منابع مورد نیازش را یکجا دریافت کند.مزیت: حلقه انتظار ایجاد نمی‌شود.معایب: اتلاف منابع، کاهش بهره‌وری.
  - عدم نگه داشتن منابع هنگام انتظار: اگر فرآیند بخواهد منبع جدید بگیرد، منابع قبلی را آزاد کند و دوباره درخواست دهد.



پیشگیری از بن‌بست Deadlock Prevention

3) **Preemption**: اجازه بدهیم منابع از فرآیند گرفته شوند (Preemption) تا به فرآیند دیگری تخصیص یابد.

- **روش‌ها:**

- اگر فرایندی منابعی را در اختیار دارد، درخواست جدیدش موافقت نشود. در این حالت باید منابع قبلی خود را آزاد کند و در صورت نیاز، مجدداً با منابع اضافی درخواست نماید.
- قبضه کردن فرایند با اولویت پایینتر: زمانی که یک فرایند نیاز به منبعی دارد که فرایند دیگری آن را نگه داشته است، سیستم عامل آنرا قبضه کرده و منابعش را آزاد میکند.
  - در فرایند های اولویت بندی شده استفاده میشود.
  - حالت منبع باید براحتی قابل ذخیره باشد تا بعدها بازیابی شود.



Deadlock Prevention پیشگیری از بن‌بست

4) **Circular Wait**: جلوگیری از شکل‌گیری حلقه انتظار.

- **روش‌ها:**
  - **ترتیب‌دهی منابع (Resource Ordering):**
    - به تمام منابع شماره یا ترتیب می‌دهیم.
    - هر فرآیند باید منابع را **به ترتیب صعودی** درخواست کند.
    - نتیجه: امکان تشکیل حلقه از بین می‌رود.
  - **نظم‌دهی استراتژیک در درخواست منابع:** هیچ فرآیندی نمی‌تواند منبع با شماره کمتر از منابع فعلی‌اش درخواست کند.



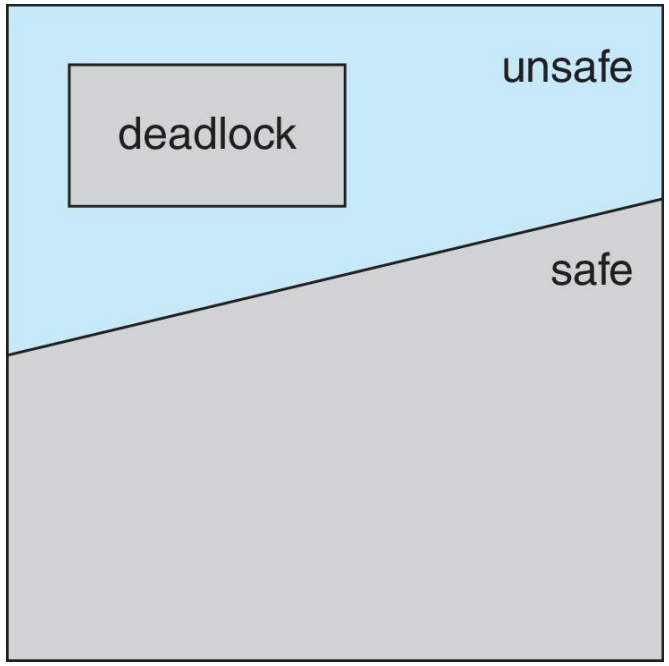
Deadlock Avoidance اجتناب از بن‌بست

- هر فرآیند **حداکثر نیاز خود** را از قبل اعلام می‌کند
- سیستم بررسی می‌کند تخصیص جدید امن هست یا نه؟
- معروف‌ترین روش: الگوریتم بانکداران **Banker's Algorithm**



حالت امن (Safe State)

- سیستم امن است اگر ترتیب اجرایی وجود داشته باشد که همه فرآیندها بدون بن‌بست تمام شوند.
- اگر یک سیستم در **حالت ایمن** (**Safe State**) باشد → بن‌بست رخ نمی‌دهد.
- اگر یک سیستم در **حالت ناایمن** (**Unsafe State**) باشد → احتمال وقوع بن‌بست وجود دارد.
- **اجتناب** (**Avoidance**) → یعنی تضمین کنیم که سیستم **هرگز وارد حالت ناایمن نشود.**





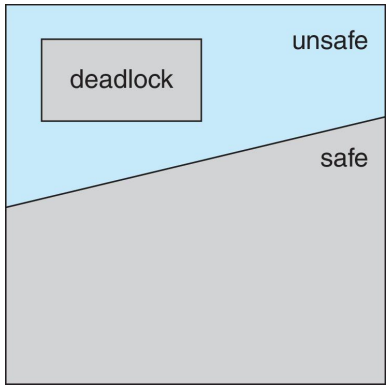
Safe, Unsafe, Deadlock State

حالت امن:

- احتمال وقوع بن‌بست وجود ندارد.
- در واقع حالتی است که در آن حداقل یک ترتیب از فرایندها وجود دارد که می‌توانند اجرا و کامل شوند، بدون اینکه بن بست رخ دهد.

حالت ناامن:

- حالتی است که امن نباشد.
- در این روش باید منابع با دقت به درخواست کننده ها ارائه شود که دچار کمبود منابع نشویم.
- احتمال وقوع بن‌بست وجود دارد.
- وضعیت بن بست، یک وضعیت ناامن است،اما تمام وضعیت های ناامن،الزاما بن‌بست نیستند، چون ممکن است یک فرایند به حداکثر منابع‌اش نیاز پیدا کند و یا حتی بخشی از منابع تخصیص یافته اش را آزاد کند.





الگوریتم اجتناب از بن‌بست

- **اگر یک منبع تنها یک نمونه داشته باشد :**

از **گراف تخصیص منابع (Resource-Allocation Graph)** برای شناسایی بن‌بست استفاده می‌کنیم.

- **اگر یک منبع چند نمونه داشته :**
- از **الگوریتم بانکدار (Banker’s Algorithm)** برای اجتناب از بن‌بست استفاده می‌کنیم.



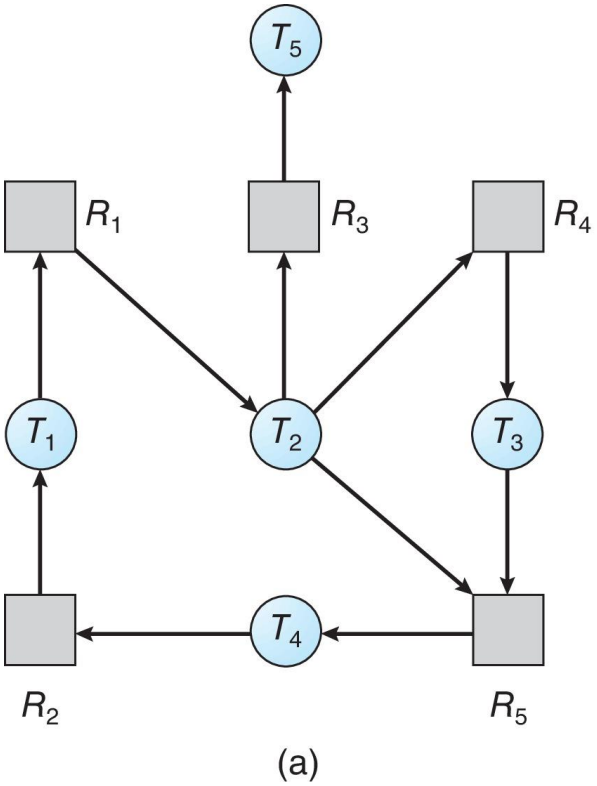
تشخیص بن‌بست با گراف انتظار

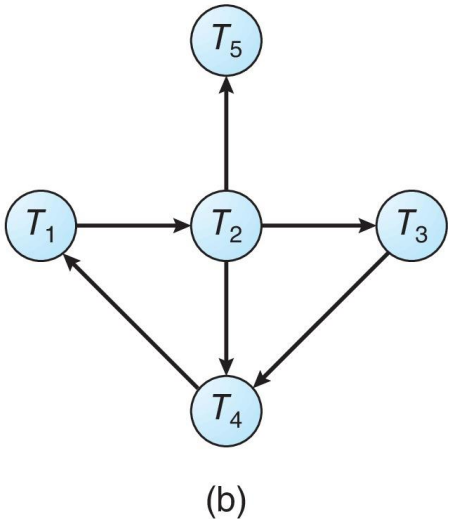
- ایجاد و نگهداری گراف انتظار (Wait-for Graph)
  - گره‌ها (Nodes): نخ‌ها (Threads) یا فرآیندها
  - یال‌ها $P_i \rightarrow P_j$ یعنی فرآیند i در انتظار منابعی است که توسط فرآیند j نگه داشته شده‌اند.
- جستجوی دوره‌ای برای تشخیص بن‌بست به صورت دوره‌ای الگوریتمی اجرا می‌شود که دنبال حلقه (Cycle) در گراف است
  - اگر حلقه‌ای وجود داشته باشد ← بن‌بست رخ داده است

وجود حلقه نشان می‌دهد که یک مجموعه از فرایندها همگی در انتظار همدیگر هستند و هیچ‌کدام نمی‌توانند ادامه دهند ← بن‌بست.



Resource-Allocation Graph and Wait-for Graph





Resource-Allocation Graph

Corresponding wait-for graph



الگوریتم بانکدار

- ارائه شده توسط Dijkstra
- هدف الگوریتم: اطمینان از اینکه سیستم هرگز وارد حالت ناایمن نشود.
- در این الگوریتم که برای اجتناب از بن‌بست استفاده می شود از بردارها (آرایه یک بعدی) و ماتریسهای(آرایه دو بعدی) زیر استفاده می شود:

نماد	توضیح
Available	بردار تعداد منابع آزاد :تعداد نمونه‌های هر نوع منبع که در حال حاضر آزاد هستند
Max	ماتریس حداکثر نیاز هر فرآیند به منابع
Allocation	ماتریس منابع تخصیص داده شده به هر فرآیند
Need	ماتریس نیاز باقی‌مانده هر فرآیند Need = Max – Allocation



روال الگوریتم بانکدار

1) سیستم درخواست منابع یک فرآیند را بررسی می‌کند.

2) اگر منابع مورد نیاز موجود باشد : فرض می‌کنیم تخصیص داده شد.

3) بررسی می‌کنیم که آیا حالت جدید ایمن است یا نه:

   1) همه فرآیندها می‌توانند منابع لازم را به ترتیب دریافت و اجرا کنند؟ → ایمن

   2) در غیر این صورت → تخصیص انجام نمی‌شود

4) تخصیص واقعی فقط اگر حالت ایمن باشد انجام می‌شود.



مثال

- با توجه به جداول زیر بگویید آیا حالت امن است؟
  - این جدول بیان میکند تنها یک نوع منبع داریم.
  - فرایند A به 6 منبع نیاز دارد و در حال حاضر 1 منبع به آن اختصاص یافته است.(برای فرایندB نیز به همین صورت)
  - در حال حاضر از کل تعداد منابع تنها 2 منبع آزاد است.(Available=2)

Available=2		
فرایند	Allocation	Max
A	1	6
B	1	5
M	2	4
S	4	7



حل مثال

- محاسبه Need:
  - Need=Max-Allocation

Available=2			
فرایند	Allocation	Max	Need
A	*1*	*6*	6-1=5
B	*1*	*5*	5-1=4
M	*2*	*4*	4-2=2
S	*4*	*7*	7-4=3



حل مثال

- پیدا کردن کوچکترین مقدارNeed:

2<=2(Available)

- تخصیص منابع مورد نیاز به فرایند مورد نظر و سپس آزاد سازی منابع تخصیص یافته فرایند یعنی تعداد allocation فرایند به تعداد available افزوده می شود

Available=2~~4~~

فرایند	Allocation	Max	Need
A	*1*	*6*	6-1=5
B	*1*	*5*	5-1=4
M	*2*	*4*	4-2=2
S	*4*	*7*	7-4=3

Available=Available + 2



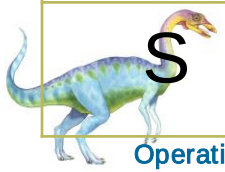
حل مثال

- پیدا کردن کوچکترین مقدارNeed:

3<4(Available)

Available=4 ~~8~~			
فرایند	Allocation	Max	Need
A	*1*	*6*	6-1=5
B	*1*	*5*	5-1=4
M	*2*	*4*	4-2=2
S	*4*	*7*	7-4=3

Available=Available + 4



حل مثال

- پیدا کردن کوچکترین مقدارNeed:

4<8(Available)

Available=8 ~~9~~			
فرایند	Allocation	Max	Need
A	*1*	*6*	6-1=5
B	*1*	*5*	5-1=4
M	*2*	*4*	4-2=2
S	*4*	*7*	7-4=3

→ Available=Available + 1



حل مثال

- پیدا کردن کوچکترین مقدارNeed:

1<9(Available)

Available=9 ~~10~~			
فرایند	Allocation	Max	Need
A	*1*	*6*	6-1=5
B	*1*	*5*	5-1=4
M	*2*	*4*	4-2=2
S	*4*	*7*	7-4=3

→ Available=Available + 1

- چون توانستیم تمام فرایند ها را حذف کنیم بنابراین مثال داده شده در صورت مسأله امن است.



الگوریتم بانکداران

- در مثال قبل فرض شد که تنها یک نوع منبع وجود دارد، اما در صورتی که انواع مختلف منبع وجود داشته باشد، نیز همانند قبل عمل میکنیم. با این تفاوت که کمترین تعداد نیاز یک سطر است.
  - بنابراین ابتدا مقدار نیاز را محاسبه میکنیم.
  - تعداد نیاز های هر فرایند را جمع میزنیم و کمترین را انتخاب میکنیم.
  - اگر سطر مربوط به کمترین مقدار از مقادیر منبع باقیمانده بیشتر بود، نتیجه میگیریم که حالت ناامن است.
  - در غیر این صورت فرایند با کمترین نیاز را حذف میکنیم و تعداد منابع تخصیص یافته آنرا را آزاد میکنیم و مراحل را دوباره تکرار میکنیم.



مثال

- آیا حالت زیر یک حالت امن است ؟

Allocation					MAX				
فرایند	R	S	T	U	فرایند	R	S	T	U
A	3	0	1	1	A	4	1	1	1
B	0	1	0	0	B	0	2	1	2
C	1	1	1	0	C	4	2	1	0
D	1	1	0	1	D	1	1	1	1
E	0	0	0	0	E	2	1	1	0

Resource	
نوع	تعداد
R	6
S	3
T	4
U	2



حل مثال

- جدول Needرا محاسبه میکنیم.

Allocation					Max				Need=max-Alloc				
فرایند	R	S	T	U	R	S	T	U	R	S	T	U	+
A	3	0	1	1	4	1	1	1	1	1	0	0	2
B	0	1	0	0	0	2	1	2	0	1	1	2	4
C	1	1	1	0	4	2	1	0	3	1	0	0	4
D	1	1	0	1	1	1	1	1	0	0	1	0	1
E	0	0	0	0	2	1	1	0	2	1	1	0	4



حل مثال

- جدول `Available` را محاسبه میکنیم.

Allocation				
فرایند	R	S	T	U
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resource	
نوع	تعداد
R	6
S	3
T	4
U	2

Available= Res[i] - A[+][i]	
نوع	تعداد
R	6-5=1
S	3-3=0
T	4-2=2
U	2-2=0



حل مثال

- با توجه به جدول Need کدام فرایند کمترین تعداد منابع را نیاز دارد؟
- چون مقدار آن از منابع باقیمانده کمتر است سطر را حذف و منابع را آزاد میکنیم.
- عملیات را برای بقیه سطر ها نیز انجام میدهیم.

Allocation					Need				
فرایند	R	S	T	U	R	S	T	U	+
A	3	0	1	1	1	1	0	0	2
B	0	1	0	0	0	1	1	2	4
C	1	1	1	0	3	1	0	0	4
D	1	1	0	1	0	0	1	0	1
E	0	0	0	0	2	1	1	0	4

Available	
نوع	تعداد
R	*1*
S	*0*
T	*2*
U	*0*



حل مثال

- در گام بعدی فرآیند A را انتخاب میکنیم و بررسی میکنیم
- Av: [2 1 2 1]+[3 0 1 1]=[5 1 3 2]

Allocation					Need				
فرایند	R	S	T	U	R	S	T	U	+
A	3	0	1	1	1	1	0	0	2
B	0	1	0	0	0	1	1	2	4
C	1	1	1	0	3	1	0	0	4
D	1	1	0	1	0	0	1	0	1
E	0	0	0	0	2	1	1	0	4

Available	
نوع	تعداد
R	2
S	1
T	2
U	1



معایب الگوریتم بانکداران:

- الگوریتم بانکداران و اجتناب از بن بست نمیتواند به طور حتم بروز بن بست را پیش بینی کند. گاهی آنرا حدس میزند و گاهی آن را اطمینان میدهد.
- فرایندها باید تمام منابع مورد نیاز خود را در ابتدا مشخص کند، در صورتی که گاهی این امر شدنی نیست!
- فرایندها باید مستقل بوده و ترتیب اجرای آنها مهم نباشد.
- تعداد منابع موجود باید ثابت باشد.



Deadlock Detection تشخیص بن‌بست

- به سیستم اجازه داده می شود وارد حالت بن بست شود و سپس تلاش می شود بن بست کشف شده و ترمیم شود. به عبارت دیگر روشی است که در آن اجازه برای انجام فرایند و هر تخصیص منبع داده میشود و سیستم عامل مرتباً وجود بن بست را بررسی میکند.

- روش حل :
  - در حالتی که از هر نوع منبع یکی موجود باشد: گراف انتظار (تخصیص منابع)
  - در حالتی که از هر نوع منبع چندتا موجود باشد: الگوریتمی شبیه بانکداران که در بازه زمانی مختلف بررسی میشود

- مزایای روش کشف بن‌بست:
  - عدم تاخیر در آغاز فرایند
  - تسهیل پردازش در حین کار



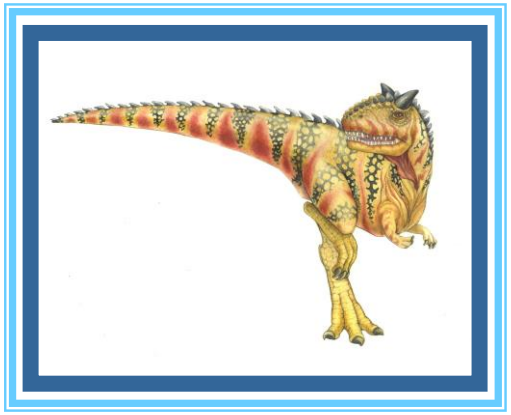
ترمیم بن‌بست

بعد از کشف بن بست، روشهای زیر برای ترمیم آن وجود دارد:

- قطع تمام فرایندهای بن بست.
- برگشت فرایندهای بن بست به نقاطی که از قبل برای بررسی تعریف شده و شروع مجدد تمام فرایندها.
- قطع پی در پی فرایندهای بن بست تاجایی که دیگر بن بست وجود نداشته باشد.
- قبضه کردن پی درپی منابع تا جایی که دیگر بن بست وجود نداشته باشد.



End of Chapter 6



Silberschatz, Galvin and Gagne ©2013